Date:

Experiment-1

a) Controlling actuators through Serial Monitor.

Software Requirements: Ardino IDE with supporting ESP-32 Board software

Hardware Requirements: ESP-32, IO Board, Connecting probes, LED,USB cable to upload the code

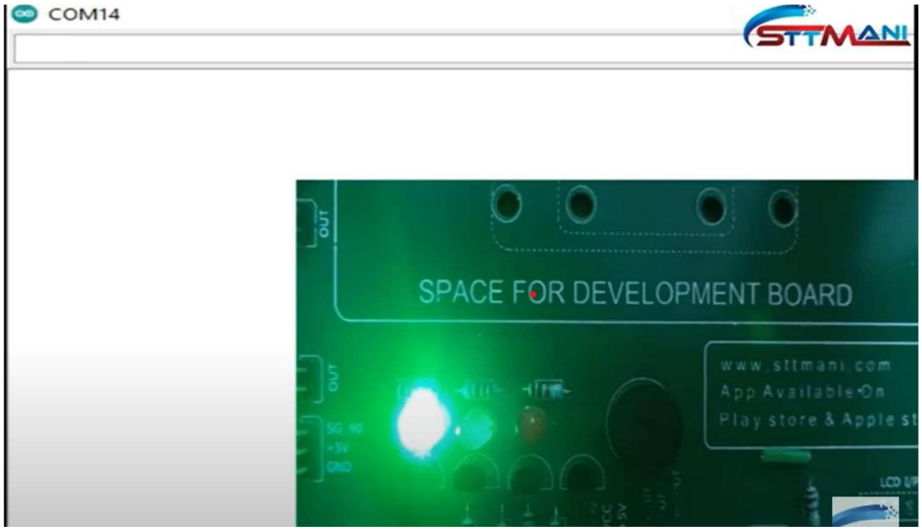
Hardware Connections:

1. Connect the USB cable to ESP-32 Board.
2. Connect the Port to PC.
3. Connect the Power Plugs.
4. Connect LED Pin to D13 Pin of ESP-32.
5. Connect Buzzer Pin to D12 Pin of ESP-32.
6. Connect The Power Supply.

Source code:

```
char input;
const int led = 13;
const int buzzer = 12;

void setup()
{
  Serial.begin(9600);
  pinMode(led, OUTPUT);
  pinMode(buzzer, OUTPUT);
  digitalWrite(led, LOW);
  digitalWrite(buzzer, LOW);
}

void loop()
{
  if(Serial.available())
  input = Serial.read();
  if(input == '0')
  digitalWrite(led, LOW);
  if(input == '1')
    digitalWrite(led, HIGH);
  if(input == '2')
  digitalWrite(buzzer, HIGH);
  if(input == '3')
  digitalWrite(buzzer, LOW);
}
```

Outcome:



The LED's on and off state is controlled using experimentation.

Date:

b) Creating different led patterns and controlling them using push button switches.

Software Requirements: Ardino IDE with supporting ESP-32 Board software [Expressif system version 2.0.9]

Hardware Requirements: Push button switches, ESP-32, Io Board, connecting probes, LED, USB cable to upload the code

Hardware Connections:

1. Connect the USB cable to ESP-32 Board.
2. Connect Green LED Pin to D12 Pin of ESP-32 Board.
3. Connect Yellow LED Pin to D13 Pin of ESP-32 Board.
4. Connect Switch 2 Pin to D23 Pin of ESP-32 Board.
5. Connect Switch 1 Pin to D22 Pin of ESP-32 Board.
6. Connect the Power Plugs.
7. Connect the Power Supply.

Source code:

```
const int led1 = 13;
const int led2 = 12;
const int led3 = 14;
const int sw1 = 23;
const int sw2 = 22;


void setup()
{
 Serial.begin(9600);
 pinMode(led1, OUTPUT);
 pinMode(led2, OUTPUT);
 pinMode(led3, OUTPUT);
 pinMode(sw1, INPUT);
 pinMode(sw2, INPUT);
 digitalWrite(led1, LOW);
 digitalWrite(led2, LOW);
 digitalWrite(led3, LOW);
}
```

```
void loop()
{
 if(digitalRead(sw1)==LOW)
 {
    digitalWrite(led1, HIGH);
    delay(300);
    digitalWrite(led1, LOW);
    delay(300);
    digitalWrite(led1, HIGH);
    delay(400);
    digitalWrite(led1, LOW);
    delay(400);
    digitalWrite(led2, HIGH);
    delay(400);
    digitalWrite(led2, LOW);
    delay(400);
    digitalWrite(led3, HIGH);
    delay(400);
    digitalWrite(led3, LOW);
    delay(400);
 }
 if(digitalRead(sw2)==LOW)
 {
    digitalWrite(led3, HIGH);
    delay(300);
    digitalWrite(led3, LOW);
    delay(300);
    digitalWrite(led3, HIGH);
    delay(400);
    digitalWrite(led3, LOW);
    delay(400);
    digitalWrite(led2, HIGH);
    delay(400);
    digitalWrite(led2, LOW);
    delay(400);
    digitalWrite(led1, HIGH);
    delay(400);
    digitalWrite(led1, LOW);
    delay(400);
 }
}
```

Outcome:

- **If push Button Switch 1 is pressed then LED's are blinked from LEFT to RIGHT on Universal board (or) Io Board**
- **If Push Button Switch 1 is pressed then LED's are blinked from RIGHT to LEFT on Universal board (or) Io Board.**

Date:

c) Controlling servo motor with the help ofjoystick.

Software Requirements:

Hardware Requirements:

Hardware Connections:

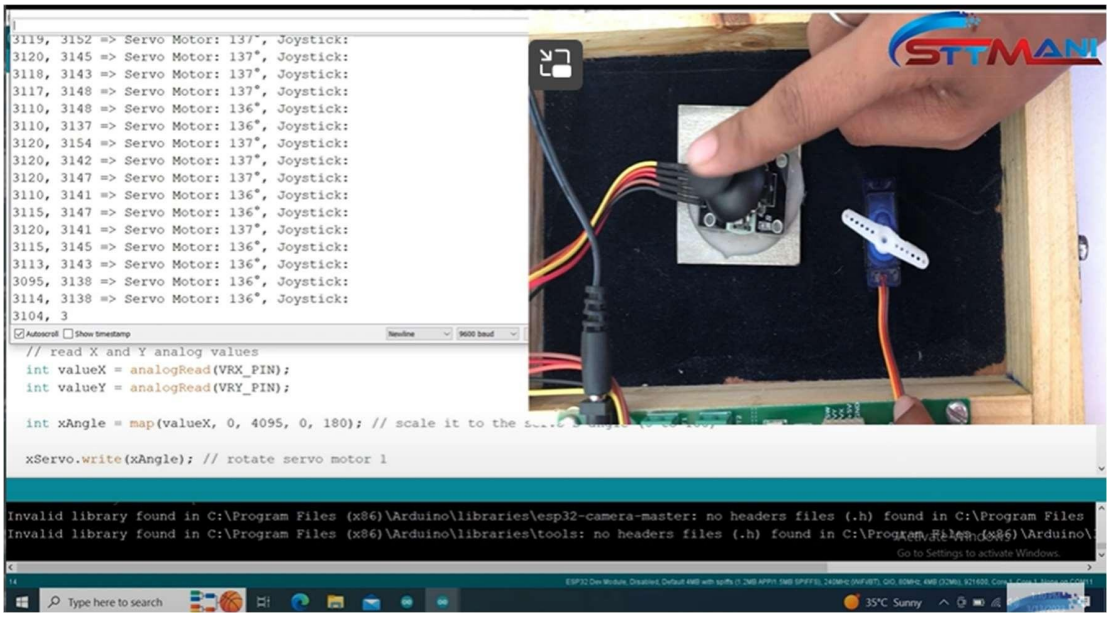
1. Connect the cable to ESP 32.
2. Connect the port to PC.
3. Connect the power plugs.
4. Connect the Joy Stick to the Board.
5. Connect Servo Brown-GND Pin.
6. Connect Servo Red ->+5V.
7. Connect Servo Yellow- SG_90
8. Connect VX Pin to VP(36) Pin of ESP 32 Board.
9. Connect the VY Pin to VN(39) Pin ESP32 Board.
10. Connect the Servp OUT Pin to D33 Pin of ESP-32.
11. Connect the Power Supply.

Source code:

```
#include <Servo.h>

#define VRX_PIN 36 // ESP32 pin GIOP36 (ADC0) connected to VRX pin

#define VRY_PIN 39 // ESP32 pin GIOP39 (ADC0) connected to VRY pin

#define SERVO_X_PIN 33 // ESP32 pin GIOP33 connected to Servo motor 1


Servo xServo; // create servo object to control a servo 1


void setup() {
```

```
  Serial.begin(9600) ;
  xServo.attach(SERVO_X_PIN);
}

void loop() {
  // read X and Y analog values
  int valueX = analogRead(VRX_PIN);
  int valueY = analogRead(VRY_PIN);

  int xAngle = map(valueX, 0, 4095, 0, 180); // scale it to the servo's angle (0 to 180)

  xServo.write(xAngle); // rotate servo motor 1

  // print data to Serial Monitor on Arduino IDE
  Serial.print("Joystick:   "); Serial.print(valueX);
  Serial.print(", ");
  Serial.print(valueY);
  Serial.print(" => Servo Motor: ");
  Serial.print(xAngle);
  Serial.print("°, ");
}
```

Outcome:


3120, 3145 => Servo Motor: 137°, Joystick:
3118, 3143 => Servo Motor: 137°, Joystick:
3117, 3148 => Servo Motor: 137°, Joystick:
3110, 3148 => Servo Motor: 136°, Joystick:
3110, 3137 => Servo Motor: 136°, Joystick:
3120, 3154 => Servo Motor: 137°, Joystick:
3120, 3142 => Servo Motor: 137°, Joystick:
3120, 3147 => Servo Motor: 137°, Joystick:
3110, 3141 => Servo Motor: 136°, Joystick:
3115, 3147 => Servo Motor: 136°, Joystick:
3120, 3141 => Servo Motor: 137°, Joystick:
3115, 3145 => Servo Motor: 136°, Joystick:
3113, 3143 => Servo Motor: 136°, Joystick:
3095, 3138 => Servo Motor: 136°, Joystick:
3114, 3138 => Servo Motor: 136°, Joystick:
3104, 3
Autoscroll
Show timestamp
// read X and Y analog values
int valueX = analogRead(VRX_PIN);
int valueY = analogRead(VRY_PIN);
int xAngle = map(valueX, 0, 4095, 0, 180); // scale it to the
xServo.write(xAngle); // rotate servo motor 1
Invalid library found in C:\Program Files (x86)\Arduino\libraries\esp32-camera-master: no headers files (.h) found in C:\Program Files
Invalid library found in C:\Program Files (x86)\Arduino\libraries\tools: no headers files (.h) found in C:\Program Files (x86)\Arduino\
Type here to search
35°C Sunny

Date:

Experiment-2

Calculate the distance to an object with the help of an ultrasonic sensor and display it on anLCD.

Software Requirements: Ardino IDE with supporting ESP-32 Board software [Express if system version 2.0.9]

Hardware Requirements: Ultrasonic sensor, LCD, ESP-32, Io Board, Connecting probes, LED, USB cable to upload the code

Hardware Connections:

1. Connect the LCD on the Development Board.
2. Connect the USB cable to ESP-32.
3. Connect the Port to pc.
4. Connect the RS(Reset) pin to D5 pin of the ESP-32.
5. Connect EN(Enable) pin to D18 pin of the ESP-32.
6. Connect D4(Data pin) to D19 PIN OF the ESP-32.
7. Connect D5(Data pin) to D21 pin of the ESP-32.
8. Connect D6(Data pin) to D22 pin of the ESP-32.
9. Connect D7(Data pin) to D23 pin of the ESP-32.
10. Connect the ECHO pin to D27 pin of ESP-32.
11. Connect TRIG pin to D33 pin of ESP-32.
12. Connect the power plugs.
13. Connect the power supply.

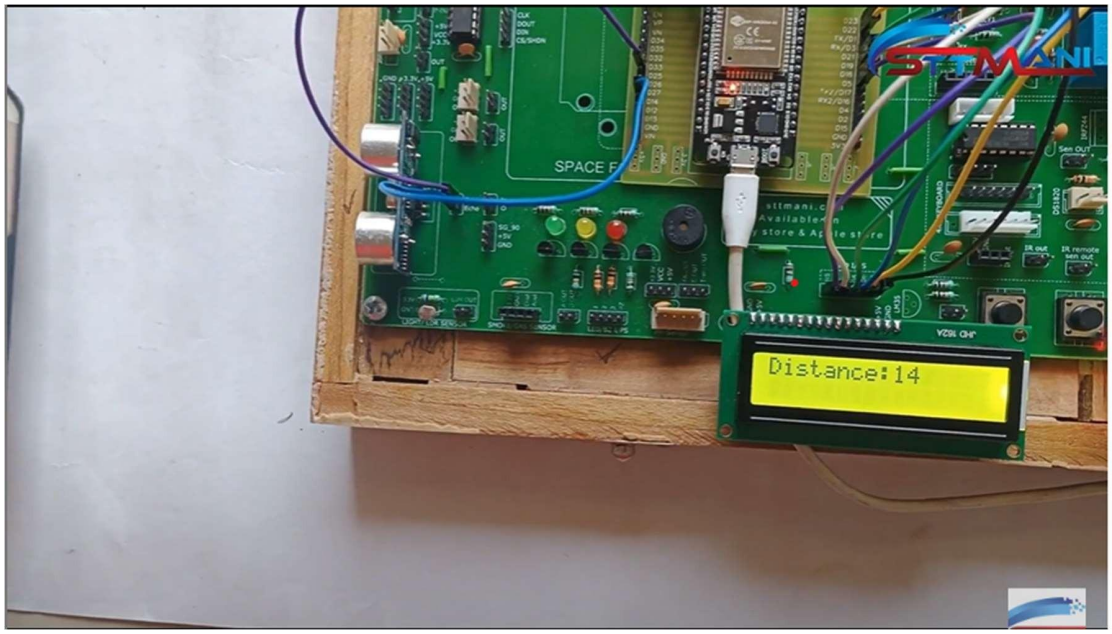
Source code:

```
#include <LiquidCrystal.h>

LiquidCrystal lcd(5,18,19,21,22,23);


const int trigPin = 33;

const int echoPin = 27;


long duration;
```

```
int distance;


void setup()
{
lcd.begin(16,2);
pinMode(trigPin, OUTPUT); // Sets the trigPin as an Output
pinMode(echoPin, INPUT); // Sets the echoPin as an Input
project_name();
}


void
loop(){ digitalWrite(trigPin,
LOW);
delayMicroseconds(2);
digitalWrite(trigPin, HIGH);
delayMicroseconds(10);
digitalWrite(trigPin, LOW);
duration = pulseIn(echoPin, HIGH);
distance= duration*0.034/2;
lcd.setCursor(0, 0);
lcd.print("Distance:       ");
lcd.setCursor(9, 0);
lcd.print(distance); lcd.setCursor(13,
0); lcd.print("Cm");
lcd.setCursor(0, 1);
```

```
lcd.print("                ");
delay(500);
}

void project_name()
{
  lcd.setCursor(0,  0);
  lcd.print("    STT-MANI    ");
  lcd.setCursor(0, 1);
  lcd.print("www.sttmani.com ");
  delay(3000);
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Measuring dist");
  lcd.setCursor(0, 1);
  lcd.print("Of An Object");
  delay(2000);
  lcd.clear();
}
```

Outcome:



By using Ultrasonic sensor an object is detected and it's distance is displayed on LCD in centimeters

Date:

Experiment-3

a) Controlling relay state based on ambient light levels using LDR sensor.

Software Requirements: Ardino IDE with supporting ESP-32 Board software [Expressif system version 2.0.9]

Hardware Requirements: LDR sensor,LCD,ESP-32, Io Board, Connecting probes, LED,USB cable to upload the code

Hardware Connections:

1. Connect the USB cable to ESP-32 Board.
2. Connect the port to PC.
3. Connect the Power Plugs.
4. Connect the LCD to the Board.
5. Connect RS(Reset) pin to D5 pin of ESP-32 Board.
6. Connect EN(Enable) pin to D18 pin of ESP-32 Board.
7. Connect D4(Data pin)to D19 pin of ESP-32 Board.
8. Connect D5(Data pin)to D21 pin of ESP-32 Board.
9. Connect D6 (Data pin)to D22 pin of ESP-32 Board.
10. Connect D7 (Data pin)to D23 pin of ESP-32 Board.
11. Connect LDR OUTPUT pin to D32 pin of ESP-32 Board.
12. Connect DC-RLY2 pin to D26 pin of ESP-32 Board.
13. Connect the Power Supply.

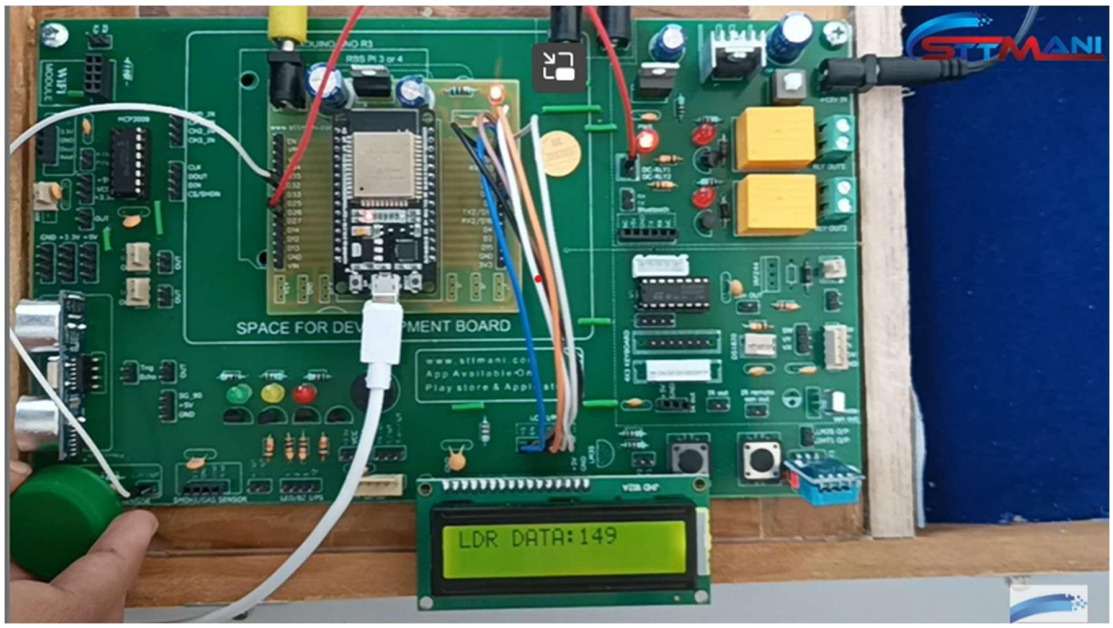
Source code:

```
#include<LiquidCrystal.h>


LiquidCrystal lcd(5, 18, 19, 21, 22, 23);


int relay=26;

void setup()

{

  lcd.begin(16, 2);
```

```
  pinMode(relay,OUTPUT);
  digitalWrite(relay,LOW);
  project();
}



void loop()
{
  int ldr_sensor_data=analogRead(32);
  lcd.setCursor(0,0);
  lcd.print("LDR DATA:       ");
  lcd.setCursor(0,1);
  lcd.print("                ");
  lcd.setCursor(9,0);
  lcd.print(ldr_sensor_data);
  delay(500);
  if(ldr_sensor_data > 4000)
  digitalWrite(relay,HIGH);
  else
  digitalWrite(relay,LOW);
  delay(500);
}



void project()
```

```
{
  lcd.setCursor(0,0);
  lcd.print(" Light Control ");
  lcd.setCursor(0,1);
  lcd.print("      By LDR    ");
  delay(3000);
  lcd.clear();
}
```

Outcome:



The "AMBIENT " light levels measured by LDR and displayed on LCD

Date:

b) Basic Burglar alarm security system with the help of PIR sensor and buzzer.

Software Requirements:

ESP 32 board-Expressif system version 2.0.9

Hardware Requirements:

ESP32, IO board, IR Sensor, LCD, Connecting probes

Hardware Connections :

1. Connect the USB Cable to ESP-32 Board.
2. Connect the port to PC.
3. Connect the Power plugs.
4. Connect the LCD to the Board.
5. Connect buzzer to the 2nd pin of ESP 32.
6. Connect IR out to the 3rd pin of ESP 32.

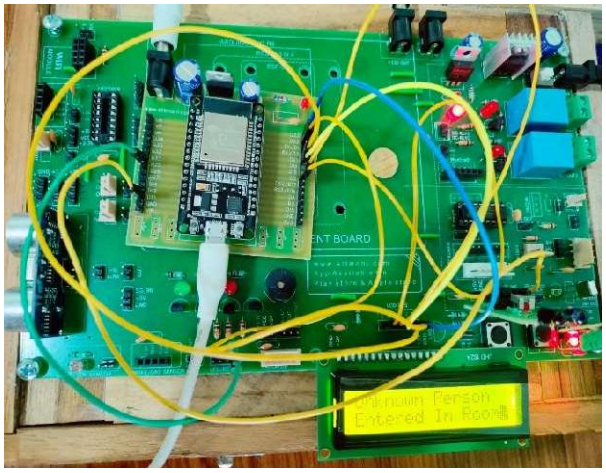
Source code:

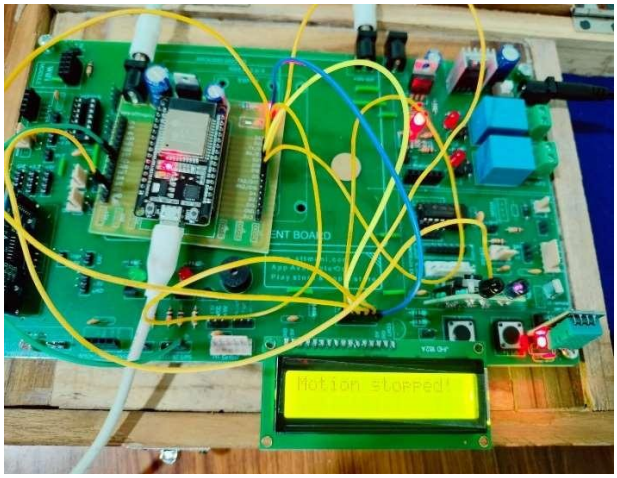
```
#include <LiquidCrystal.h>

#define PIR_sensor_value 13Qq20060suioplj;.,

#define buzzer 25

LiquidCrystal lcd(5, 18, 19, 21, 22, 23);

int sensor_value = 0;

void setup()

{

 lcd.begin(16, 2);

 pinMode(buzzer, OUTPUT);

 digitalWrite(buzzer, LOW);
```

```
  pinMode(PIR_sensor_value, INPUT);
  lcd.setCursor(0, 0);
  lcd.print("Burglar Alaram ");
  lcd.setCursor(0, 1);
  lcd.print("Security By PIR");
  delay(1000);
}

void loop()
{
  if (digitalRead(PIR_sensor_value) == HIGH)
  {
    lcd.setCursor(0, 0);
    lcd.print("Unknown Person ");
    lcd.setCursor(0, 1);
    lcd.print("Entered In Room");
    buzzer_sound();
  }
  else
  {
    lcd.setCursor(0, 0);
    lcd.print("Motion stopped! ");
    lcd.setCursor(0,   1);
    lcd.print("                ");
    delay(50);
```

```
  }
}

void buzzer_sound()
{
  digitalWrite(buzzer, HIGH);
  delay(600);
  digitalWrite(buzzer, LOW);
  delay(400);
  digitalWrite(buzzer, HIGH);
  delay(600);
  digitalWrite(buzzer, LOW);
  delay(400);
  digitalWrite(buzzer, HIGH);
  delay(600);
  digitalWrite(buzzer, LOW);
  delay(400);
}
```

Outcome:





If IR sensor detects any object movement then "unknown person entered into room" message is displayed on LCD and the Buzzer rings otherwise " Motion stopped " message is displayed on LCD

Date:

C) Displaying humidity and temperature values on LCD

Software Requirements: Ardino IDE with supporting ESP-32 Board software

Hardware Requirements:DHT 11 sensor, LDC, ESP-32, Io Board, Connecting probes, LED, USB cable to upload the code.

Hardware Connections:

1. Connect the USB Cable to ESP-32 Board.
2. Connect the port to PC.
3. Connect the Power plugs.
4. Connect the LCD to the Board.
5. Connect the RS(Reset) pin to D5 pin of ESP-32 Board.
6. Connect the EN(Enable) pin to the D18 pin of ESP-32 Board.
7. Connect the D4(Data Pin)to D19 pin of ESP-32Board.
8. Connect the D5(Data pin)to D21 pin of ESP-32 board.
9. Connect the D6(Data pin)to D22 pin of ESP-32 board.
10. Connect the D7(Data pin)to D23 pin of ESP-32 Board.
11. Connect the DHT11 OUT pin to D33 pin of ESP-32 Board.
12. Connect the Power Supply.

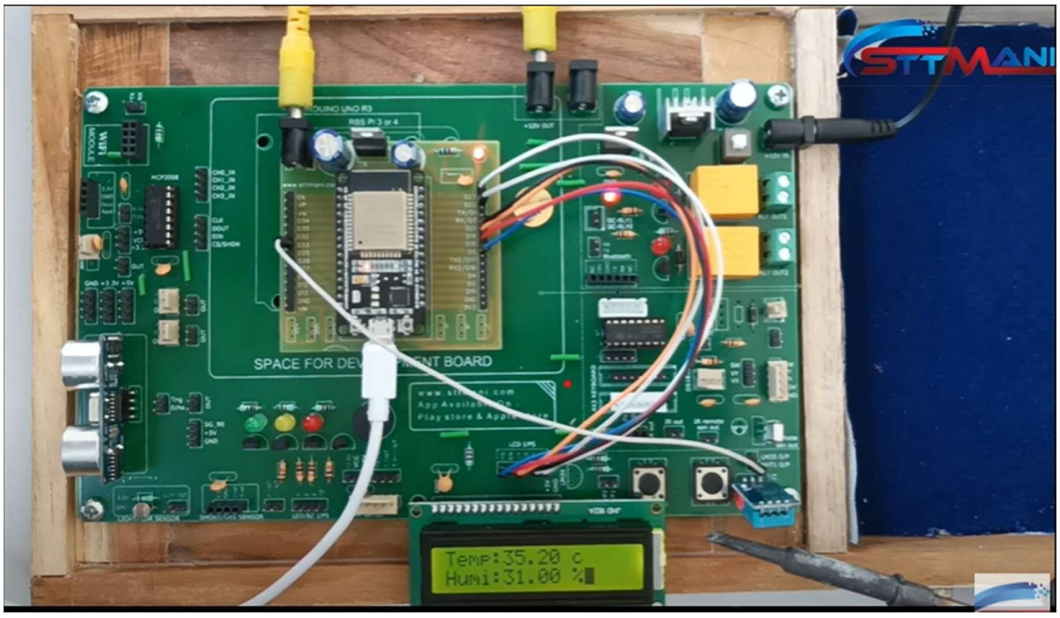
Source code:

```
#include<LiquidCrystal.h>
#include  "DHT.h"
#include <Wire.h>
#define DHT11PIN 33
LiquidCrystal lcd(5, 18, 19, 21, 22, 23);
DHT dht(DHT11PIN, DHT11);
void setup()
{
```

```
  lcd.begin(16,2);
  dht.begin();
  project_name();
}

void loop()
{
  float humi = dht.readHumidity();
  float temp = dht.readTemperature();
  lcd.setCursor(0,0);
  lcd.print("Temperature: ");
  lcd.setCursor(12,0);  lcd.print(temp);
  lcd.setCursor(0,1);
  lcd.print("Humidity: ");
  lcd.setCursor(9,1);
  lcd.print(humi);
  lcd.print("%");
  delay(1000);
}
void project_name()
{
  lcd.setCursor(0,0);
  lcd.print("Temperature and ");
  lcd.setCursor(0,1);
```

```
  lcd.print("Humidity Display");
  delay(2000);
  lcd.clear();
}
```

Outcome:



The measured Temperature and Humidity values by the DHT 11 sensor are displayed on LDC

Date:

C) Controlling Servo Motor Using Joystick on Serial Monitor Interfacing with ESP32

Aim: Controlling Servo Motor Using Joystick on Serial Monitor Interfacing with ESP32

Software Requirements: Arduino IDE with supporting ESP32 board software [expressif system version 2.0.9]

Hardware Requirements: Development Board, ESP32 board, Servo Motor, Joystick, 12V Adaptor, Power jack, USB Cable, Jumper Wires

Hardware Connections:

1. Connect the cable to ESP32
2. Connect the port to PC.
3. Connect the Power Plugs
4. Connect the Joystick to the board.
5. Connect servo Brown-GND pin
6. Connect servo RED -> +5v
7. Connect servo YELLOW -> SG_90\
8. Connect VX pin to VP (36) pin of ESP32 board
9. Connect the VY pin to VN (39) pin of ESP32 board
10. Connect the servo OUT pin to D33 pin of ESP32
11. Connect the power supply.

Source code:

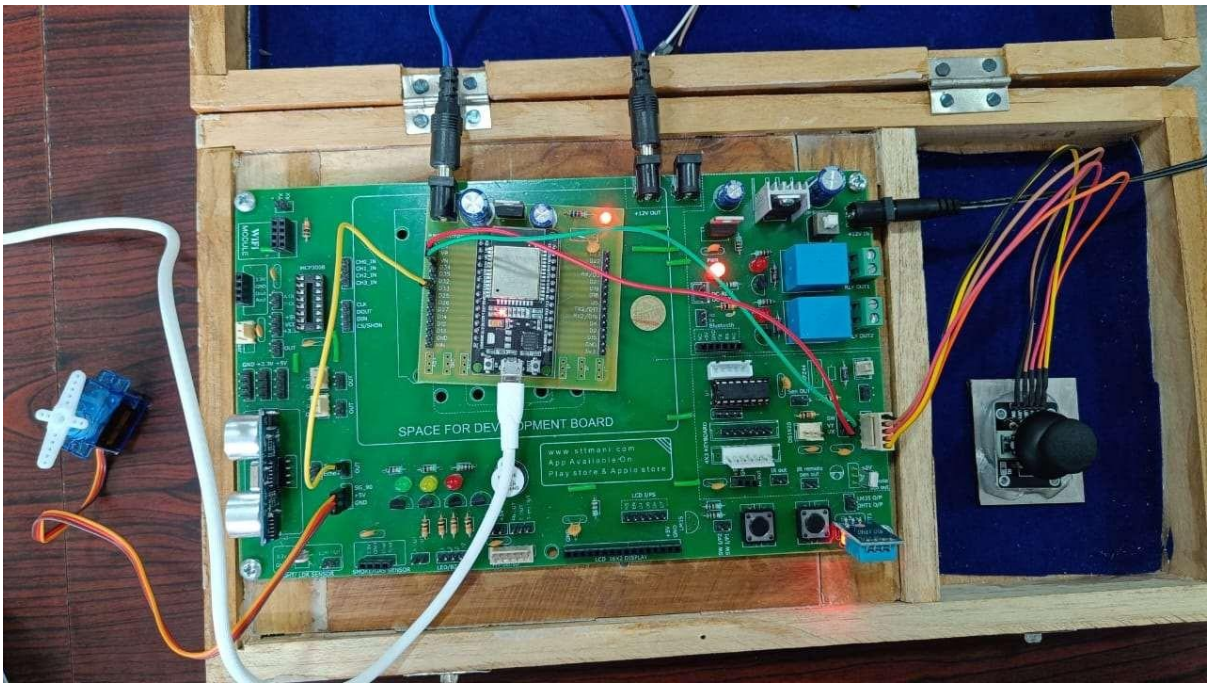
```
#include <Servo.h>

#define VRX_PIN     36 // ESP32 pin GIOP36 (ADC0) connected to VRX pin

#define VRY_PIN     39 // ESP32 pin GIOP39 (ADC0) connected to VRY pin

#define SERVO_X_PIN 33 // ESP32 pin GIOP33 connected to Servo motor 1

Servo xServo; // create servo object to control a servo 1

void setup() {
```

```
Serial.begin(9600) ;
xServo.attach(SERVO_X_PIN);
}
void loop() {
 // read X and Y analog values
 int valueX = analogRead(VRX_PIN);
 int valueY = analogRead(VRY_PIN);
 int xAngle = map(valueX, 0, 4095, 0, 180); // scale it to the servo's angle (0 to 180)
 xServo.write(xAngle); // rotate servo motor 1
 // print data to Serial Monitor on Arduino IDE
 Serial.println("Joystick:   ");
 Serial.print(valueX);
 Serial.print(", ");
 Serial.print(valueY);
 Serial.print(" => Servo Motor: ");
 Serial.print(xAngle);
 Serial.print("°, ");
}
```

Precautions:

1. Take care about given power supply (12 v)
2. Jumper wires given carefully whenever given circuit connection

Outcome:



Servo motor is controlled using Joystick

Date:

Experiment-4

a) Controlling relay state based on input from IR sensor interfacing with Raspberry pi Pico.

Aim: Controlling relay state based on input from IR sensor interfacing with Raspberry pi Pico

Hardware Requirements: Development Board, Raspberry pi Pico board, 12v Adapter, IR sensor, Relay, Power jack, USB cable, Jumper Wires

Software Requirements: Thonny IDE with supporting Raspberry pi Pico Board software

Hardware Connections:

1. IR sensor out pin is connected to Raspberry pi Pico GPIO pin 3
2. Relay out pin is connected to Raspberry pi Pico GPIO pin 2
3. Power jack is connected to Raspberry pi Pico
4. Insert IR sensor in IR Jack
5. USB connector is connected to Raspberry pi Pico to CPU/Laptop
6. Connect the 12v power supply to development board
7. Switch on the power supply.

Software Connections:

1. Click on Thonny IDE
2. Click on file
3. Click on New
4. Write a program as per pin connection of circuit
5. Click on save
6. Click on Run, select interpreter and select micro python (Raspberry pi Pico), select port then click OK
7. Click on Run

Source code:

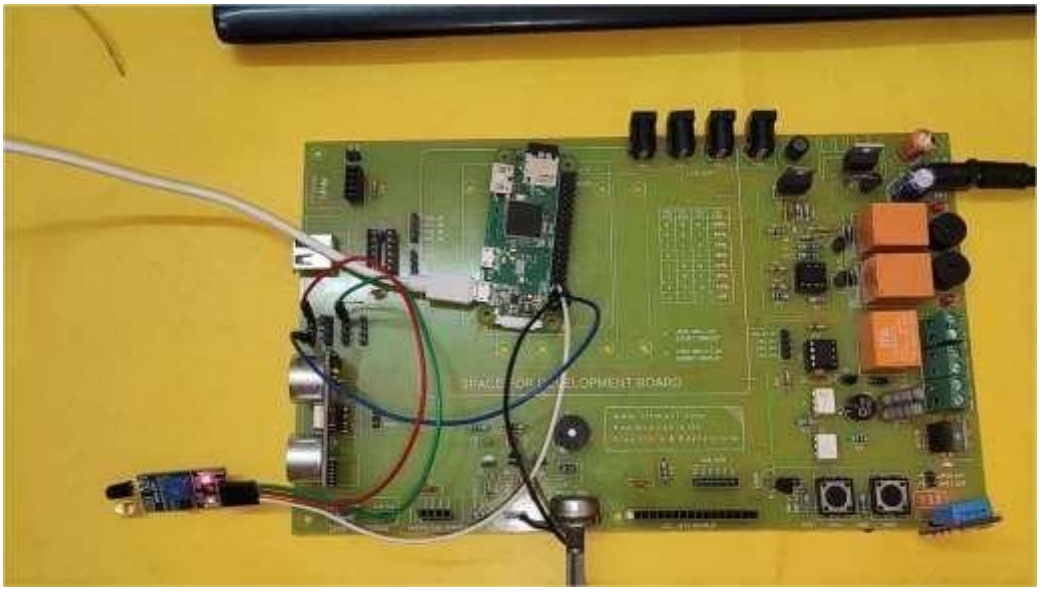
```
import machine
import time

relay_pin = machine.Pin(2, machine.Pin.OUT)
```

```
ir_sensor_pin = machine.Pin(3, machine.Pin.IN, machine.Pin.PULL_DOWN)

while True:
    if ir_sensor_pin.value() == 0:
        relay_pin.on()
        print('Relay ON')
        time.sleep(2)
    else:
        relay_pin.off()
        print('Relay OFF')
    time.sleep(0.1)
```

Precautions:

1. Take care about given power supply (12 v)
2. Jumper wires given carefully whenever given circuit connection

Outcome:



Relay is controlled by using IR sensor with Raspberry pi Pico

Date:

b) Interfacing with stepper motor with Raspberry pi Pico.

Aim: Interfacing stepper motor with Raspberry pi

Hardware Requirements: Development Board, Raspberry pi Pico board, 12 v Adaptor, stepper motor, Power jack, USB cable, Jumper Wires.

Software Requirements: Thonny IDE with supporting Raspberry pi Pico board software.

Hardware Connections:

1. Stepper motor sensor out pins are connected to Raspberry pi Pico GPIO pins 0,1,2,3
2. Power jack is connected to the Raspberry pi Pico
3. Insert stepper motor sensor in stepper motor jack
4. USB connector is connected to Raspberry pi Pico to CPU/Laptop
5. Connect the 12v power supply to development board
6. Switch on the power supply

Software Connections:

1. Click on Thonny IDE
2. Click on file
3. Click on New
4. Write a program as per circuit pin connections
5. Click on save
6. Click on Run, select interpreter and select Micro-python (Raspberry pi Pico), select port then click OK
7. Click on Run

Source code:

```
import machine

import utime


# Define motor pins

IN1 = machine.Pin(0, machine.Pin.OUT)
```

```
IN2 = machine.Pin(1, machine.Pin.OUT)
IN3 = machine.Pin(2, machine.Pin.OUT)
IN4 = machine.Pin(3, machine.Pin.OUT)
# Define motor steps and delay time
steps = [[1,0,0,1],[1,0,0,0],[1,1,0,0],[0,1,0,0],[0,1,1,0],[0,0,1,0],[0,0,1,1],[0,0,0,1]]
delay = 3
 # Define motor function
def motor(step, direction):
   for i in range(0, 512):
      for j in range(0, 8):
         IN1.value(steps[j][0]^direction)
         IN2.value(steps[j][1]^direction)
         IN3.value(steps[j][2]^direction)
         IN4.value(steps[j][3]^direction)
         utime.sleep_ms(delay)
# Test motor
while True:
   motor(steps, 0)
   utime.sleep(3)
   motor(steps, 1)
   utime.sleep(3)
```

Precautions:

1. Take care about given power supply (12 v)
2. Jumper wires given carefully whenever given circuit connection

Outcome:

Interfaced Stepper Motor sensor with Raspberry pi Pico starts rotating.

Date:

c) Advanced Burglar alarm security system with the help of PIR sensor, buzzer and keypad interfacing with Raspberry pi Pico.

Aim: Advanced burglar alarm security system with the help of PIR sensor, buzzer and keypad.

Hardware Requirements: Development Board, 12v Adaptor, Raspberry pi Pico board, 4X3 keypad, Buzzer, PIR sensor, Power jack, USB cable, Jumper Wires.

Software Requirements: Thonny IDE with supporting Raspberry pi Pico board software.

Hardware Connections:

1. Keypad sensor out pins R1, R2, R3, R4, C1, C2, C3 are connected to Raspberry pi Pico GPIO pins 9, 8, 7, 6, 5, 4, 3. Buzzer, PIR and LED out pins are connected to Raspberry pi Pico GPIO pins 16, 15, 17
2. Power jack is connected to the Raspberry pi Pico
3. Insert keypad, PIR sensor in keypad, PIR jacks
4. USB connection is connected to Raspberry pi Pico to CPU/Laptop
5. Connect the 12v power supply to development board
6. Switch on the power supply

Software Connections:

1. Click on Thonny IDE
2. Click on file
3. Click on New
4. Write a program as per circuit pin connections
5. Click on save
6. Click on Run, select interpreter and select Micro-python (Raspberry pi Pico), select port then click Ok
7. Click on Run

Source code:

```
# Matrix Keypad
# NerdCave - https://www.youtube.com/channel/UCxxs1zIA4cDEBZAHIJ80NVg
from machine import Pin
import utime
# Define PIR sensor pin and buzzer pin
PIR_PIN = machine.Pin(15, machine.Pin.IN)
BUZZER_PIN  = machine.Pin(16,  machine.Pin.OUT)
# Create a map between keypad buttons and characters
matrix_keys = [['1', '2', '3'],
               ['4', '5', '6'],
               ['7', '8', '9'],
               ['*', '0', '#']]
# Define PINs according to cabling
keypad_rows = [9,8,7,6]
keypad_columns = [5,4,3,2]
col_pins = []
row_pins = []
## the keys entered by the user
guess = []
#our secret pin, shhh do not tell anyone
secret_pin = ['1','2','3','4','5','6']
#setup pin to be an output
led = Pin(17, Pin.OUT, Pin.PULL_UP)
for x in range(0,4):
    row_pins.append(Pin(keypad_rows[x], Pin.OUT))
    row_pins[x].value(1)
```

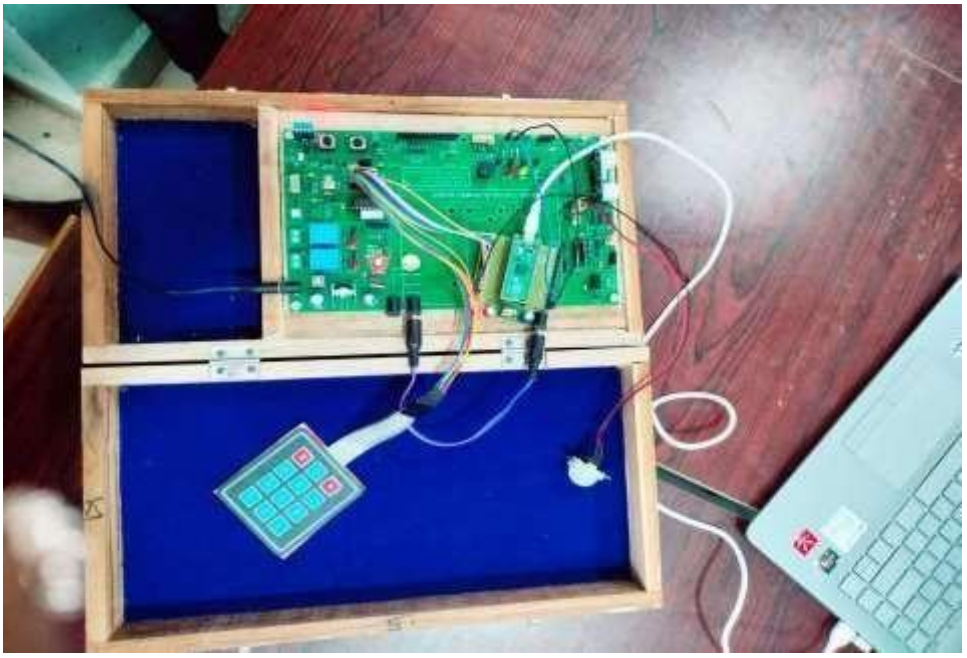
```
    col_pins.append(Pin(keypad_columns[x], Pin.IN, Pin.PULL_DOWN))
    col_pins[x].value(0)
    ################################Scan keys ####################
  def scankeys():
      for row in range(4):
      for col in range(3):
        row_pins[row].high()
        key = None
                if col_pins[col].value() == 1:
          print("You have pressed:", matrix_keys[row][col])
          key_press = matrix_keys[row][col] utime.sleep(0.3)
          guess.append(key_press)
        if len(guess) == 6:
          checkPin(guess)
                    for x in range(0,6):
            guess.pop()
                  row_pins[row].low()
  ################################To check Pin #################
def checkPin(guess):
  if guess == secret_pin:
          print("You got the secret pin correct")
      led.value(1)
      utime.sleep(3)
      led.value(0)
        else:
      print("Better luck next time")
```

```
        BUZZER_PIN.value(1)
        utime.sleep(0.5)
        BUZZER_PIN.value(0)
        utime.sleep(0.5)
        BUZZER_PIN.value(1)
        utime.sleep(0.5)
        BUZZER_PIN.value(0)
        utime.sleep(0.5)
################################################################
        # Define alarm function
def activate_alarm():
    while True:
        if PIR_PIN.value() == 1:
            BUZZER_PIN.value(1)
            utime.sleep(0.5)
            BUZZER_PIN.value(0)
            utime.sleep(0.5)
    print("Enter the secret Pin")
while True:
        scankeys()
    # Activate alarm if PIR sensor detects motion
    if PIR_PIN.value() == 1:
        print('Intruder detected')
        BUZZER_PIN.value(1)
        activate_alarm()
    utime.sleep(0.1)
```

Precautions:

1. Take care about given power supply (12 v)
2. Jumper wires given carefully whenever given circuit connection

Outcome:



PIR sensor, buzzer and keypad interfaced with Raspberry pi Pico.

Date:

d) Automated LED light control based on input from PIR (to detect if people are present) & LDR (ambient light level) interfacing with Raspberry pi Pico.

Aim: TO interface Automated LED light control based on input from PIR and LDR with Raspberry pi Pico board.

Hardware Requirements: Development Board, Raspberry pi Pico board, 12v Adapter, LED, LDR sensor, PIR sensor, Power jack, USB cable, Jumper Wires.

Software Requirements: Thonny IDE with supporting Raspberry pi Pico board software.

Hardware Connections:

1. PIR, LDR and LED out pins are connected to Raspberry pi Pico GPIO pins 0, 26, 1
2. Power jack is connected to Raspberry pi Pico
3. Insert PIR sensor in PIR jack
4. USB connector is connected to Raspberry pi Pico to CPU/Laptop
5. Connect the 12v power supply to development board
6. Switch on the power supply

Software Connections:

1. Click on Thonny IDE
2. Click on file
3. Click on New
4. Write a program as per circuit pin connections
5. Click on save
6. Click on Run, select interpreter and select Micro-python (Raspberry pi Pico), select port then click OK
7. Click on Run

Source code:

```
import machine
import utime
# Set up PIR sensor input pin
pir_pin = machine.Pin(0, machine.Pin.IN)
# Set up LDR sensor input pin
ldr_pin = machine.ADC(26)
# Set up LED output pin
led_pin = machine.Pin(1, machine.Pin.OUT)
while True:
    # Read PIR sensor input
    pir_input = pir_pin.value()
    # Read LDR sensor input
    ldr_input = ldr_pin.read_u16()
    # Adjust LED output based on PIR and LDR inputs
    if pir_input == 1 and ldr_input < 5000:
        # If someone is present and it's dark, turn on LED
        led_pin.on()
    else:
        # Otherwise, turn off LED
        led_pin.off()
    # Wait a short period of time before checking inputs again
  utime.sleep(0.1)
```

Precautions:

1. Take care about given power supply (12 v)
2. Jumper wires given carefully whenever given circuit connection

Outcome:

LED is controlled by using LDR and PIR sensors with Raspberry pi Pico.

Date:

Experiment-5

Upload humidity & temperature data to ThingSpeak, periodically logging ambient light level to ThingSpeak

Aim: To upload humidity and temperature data on a web-based application interfacing with ESP32 board

Hardware Requirements: Development Board, ESP32 board, DHT11 sensor, 12v Adaptor, Power jack, USB cable, Jumper Wires.

Software Requirements: Arduino IDE with supporting ESP32 Board software.

Hardware Connections:
1. DHT11 sensor is Development Board DHT11 out pin connected to D33 pin of ESP32 board.
2. Power jack is connected to the ESP32 board
3. USB connector is connected to ESP32 board and CPU
4. Connect the 12v power supply to development board.
5. Switch on power supply.

Software Connections:
1. Click on Arduino IDE
2. Click on file
3. Click on New
4. Write a program as per circuit pin connections
5. Click on save
6. Click on verify
7. Click on tools select ESP32 board and select port
8. Click on upload the code into ESP32 board by using USB cable
9. GOTO tools click on serial plotter verify the output in graph format

Source code:

```
#include <WiFi.h>

#include <Wire.h>
```

```
#include "DHT.h"
#define DHTPIN 33
float temp;
float humi;
// Uncomment whatever type you're using!
#define DHTTYPE DHT11     // DHT 11
// Replace with your network credentials
const char* ssid     = "STTMANI";
const char* password = "hailucky123,./";
// Set web server port number to 80
WiFiServer server(80);
// Variable to store the HTTP request
String header;
DHT dht(DHTPIN, DHTTYPE);
void setup()
{
  Serial.begin(115200);
  dht.begin();
  bool status;
  // Connect to Wi-Fi network with SSID and password
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED)
    {delay(1000);
```

```
    Serial.print(".");
  }
  // Print local IP address and start web server
  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
  server.begin();
}
void loop()
{
  humi = dht.readHumidity();
  temp = dht.readTemperature();
  WiFiClient client = server.available(); // Listen for incoming clients
  if (client)
  { // If a new client connects,
    Serial.println("New Client.");          // print a message out in the serial port
    String currentLine = "";                // make a String to hold incoming data from the
client
    while (client.connected()) {            //  loop while the client's  connected
      if (client.available()) {             // if there's bytes to read from the client,
      char c = client.read();               // read a byte, then
        Serial.write(c);                    // print it out the serial monitor
        header += c;
        if (c == '\n') {                    // if the byte is a newline character
```

```
            // if the current line is blank, you got two newline characters in a row.
            // that's the end of the client HTTP request, so send a response:
            if (currentLine.length() == 0) {
              // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
              // and a content-type so the client knows what's coming, then a blank line:
              client.println("HTTP/1.1 200 OK");
              client.println("Content-type:text/html");
              client.println("Connection: close");
              client.println();
              // Display the HTML web page
              client.println("<!DOCTYPE html><html>");
              client.println("<head><meta name=\"viewport\" content=\"width=device-width,
initial-scale=1\">");
              client.println("<link rel=\"icon\" href=\"data:,\">");
              // CSS to style the table
              client.println("<style>body { text-align: center; font-family: \"Trebuchet MS\",
Arial;}");
              client.println("table { border-collapse: collapse; width:35%; margin-left:auto;
margin-right:auto; }");
              client.println("th { padding: 12px; background-color: #0043af; color: white; }");
              client.println("tr { border: 1px solid #ddd; padding: 12px; }");
              client.println("tr:hover { background-color: #bcbcbc; }");
              client.println("td { border: none; padding: 12px; }");
              client.println(".sensor { color:white; font-weight: bold; background-color:
#bcbcbc; padding: 1px; }");
              // Web Page Heading
              client.println("</style></head><body><h1>DHT11 Using ESP32</font></h1>");
```

```
            client.println("<table><tr><th>SENSOR</th><th>VALUE</th></tr>");
            client.println("<tr><td>Temperature</td><td><span class=\"sensor\">");
            client.println(temp);
            client.println("</span></td></tr>");
            client.println("<tr><td>Humidity</td><td><span class=\"sensor\">");
            client.println(humi);
            client.println("</span></td></tr>");
            client.println("</body></html>");
            // The HTTP response ends with another blank line
            client.println();
            // Break out of the while loop
            break;
          } else { // if you got a newline, then clear currentLine
            currentLine = "";
          }
        } else if (c != '\r') { // if you got anything else but a carriage return character,
          currentLine += c;      // add it to the end of the currentLine
        }
      }
    }
    // Clear the header variable
    header = "";
    // Close the connection
    client.stop();
    Serial.println("Client disconnected.");
```

```
        Serial.println("");
      }
    }
```

Precautions:

1. Take care about given power supply (12 v)
2. Jumper wires given carefully whenever given circuit connection

Outcome:

DHT11 is controlled by using web-based application with ESP32

Date:

Experiment - 6

Controlling LED, Buzzer & Relay using BLYNK APP interfacing with ESP32

Aim: Controlling LED, Buzzer and Relay using BLYNK app interfacing with ESP32 board.

Hardware Requirements: Development Board, LED, ESP32 board, Relay, Buzzer, 12v Adapter, Power jack, USB cable, Jumper Wires.

Software Requirements: Arduino IDE with supporting with ESP32 board software.

Hardware Connections:

1. LED, Buzzer, Relay in Development board LED, BZ, DC-RLY1 pins connected to D13
2. Power jack is connected to the ESP32 board
3. USB connector is connected to ESP32 board and CPU
4. Connect the 12v power supply to development board
5. Upload corresponding program into ESP32 board
6. Switch on Power supply

Software Connections:

1. Click on Arduino IDE
2. Click on file
3. Click on New
4. Write a program as per circuit pin connections
5. Click on save
6. Click on verify
7. Click on tools select ESP32 board and select port
8. Click on the upload the code into ESP32 board by using USB cable
9. GOTO tools click on serial plotter verify the output in graph format

Source code:

```
#define BLYNK_TEMPLATE_ID "TMPL303Hs8kGk"
```

```
#define BLYNK_DEVICE_NAME "Blink App Control"
#define BLYNK_AUTH_TOKEN "s_-I5drDSeJE6yVAj6-Uj1nX0OnMm3fA"
 // Comment this out to disable prints and save space
#define BLYNK_PRINT Serial
 #include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>
 char auth[] = BLYNK_AUTH_TOKEN;
 // Your WiFi credentials.
// Set password to "" for open networks.
char ssid[] = "STTMANI";
char pass[] = "hailucky123,./";
 BlynkTimer timer;
 BLYNK_WRITE(V0)
{
  int pinValue = param.asInt();
  digitalWrite(13, pinValue);
}
BLYNK_WRITE(V1)
{
  int pinValue = param.asInt();
  digitalWrite(12, pinValue);
}
BLYNK_WRITE(V2)
{
```

```
  int pinValue = param.asInt();
  digitalWrite(14, pinValue);
}
void setup()
{
  pinMode(13, OUTPUT);
  pinMode(12, OUTPUT);
  pinMode(14, OUTPUT);
  Serial.begin(115200);
  delay(100); Blynk.begin(auth,
  ssid, pass);
}
 void loop()
{
  Blynk.run();
}
```

Precautions:

1. Take care about given power supply (12 v)
2. Jumper wires given carefully whenever given circuit connection

Outcome:

LED, Buzzer and Relay are controlled by using BLYNK app with ESP32

Date:

Experiment - 7

Controlling LED, Buzzer & Relay using web-based application interfacing with ESP32.

Aim: To interface controlled LED, Buzzer and Relay on LCD interfacing with ESP32 board.

Hardware Requirements: Development Board, LED, ESP32 board, Relay, Buzzer, 12v Adapter, Power jack, USB cable, Jumper Wires.

Software Requirements:

1. LED, Buzzer, Relay in Development Board, LED1, LED2, BZ, DC-RLY1 pins connected to D13, D14, D27 pins of ESP32 board, According to program. LCD pins connected to ESP32 board

Hardware Connections:

1. LED, Buzzer, Relay in Development board, LED1, LED2, BZ, DC-RLY1 pins connected to D13, D12, D14, D27 pins of ESP32 board, According to program LCD pins connected to ESP32 board
2. Power jack is connected to ESP32 board
3. USB connector is connected to ESP32 board
4. Connect the 12v power supply to development board
5. Upload corresponding program into ESP32 board
6. Switch on power supply

Software Connections:

1. Click on Arduino IDE
2. Click on file
3. Click on New
4. Write a program as per circuit pin connections
5. Click on save
6. Click on verify
7. Click on tools select ESP32 board and select port
8. Click on upload the code into ESP32 board by using USB cable
9. GOTO tools click on serial plotter verify the output in graph format

Source code:

```
// Import required libraries
#include <WiFi.h>
#include <AsyncTCP.h>
#include <ESPAsyncWebServer.h>
// Replace with your network credentials
const char* ssid = "STTMANI";
const char* password = "hailucky123,./";
const char* PARAM_INPUT_1 = "output";
const char* PARAM_INPUT_2 = "state";
// Create AsyncWebServer object on port 80
AsyncWebServer server(80);
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE HTML><html>
<head>
  <title>ESP Web Server</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" href="data:,">
  <style>
    html {font-family: Arial; display: inline-block; text-align: center;}
    h2 {font-size: 3.0rem;}
    p {font-size: 3.0rem;}
    body {max-width: 900px; margin:0px auto; padding-bottom: 25px;}
    .switch {position: relative; display: inline-block; width: 120px; height: 68px}
    .switch input {display: none}
```

```
   .slider {position: absolute; top: 0; left: 0; right: 0; bottom: 0; background-color: #ccc; border-radius: 6px}

   .slider:before {position: absolute; content: ""; height: 52px; width: 52px; left: 8px; bottom: 8px; background-color: #fff; -webkit-transition: .4s; transition: .4s; border-radius: 3px}

   input:checked+.slider {background-color: #b30000}

   input:checked+.slider:before {-webkit-transform: translateX(52px); -ms-transform: translateX(52px); transform: translateX(52px)}

  </style>

</head>

<body>

  <h2>ESP Web Server</h2>

  %BUTTONPLACEHOLDER%

<script>function toggleCheckbox(element)

  {var xhr = new XMLHttpRequest();

  if(element.checked){ xhr.open("GET", "/update?output="+element.id+"&state=1", true); }

  else { xhr.open("GET", "/update?output="+element.id+"&state=0", true); }

  xhr.send();

}

</script>

</body>

</html>

)rawliteral";

// Replaces placeholder with button section in your web page

String processor(const String& var){

  //Serial.println(var);
```

```
  if(var == "BUTTONPLACEHOLDER"){

    String buttons = "";

    buttons += "<h4>Output - GPIO 13</h4><label class=\"switch\"><input
type=\"checkbox\" onchange=\"toggleCheckbox(this)\" id=\"13\" " + outputState(2) +
"><span class=\"slider\"></span></label>";

    buttons += "<h4>Output - GPIO 12</h4><label class=\"switch\"><input
type=\"checkbox\" onchange=\"toggleCheckbox(this)\" id=\"12\" " + outputState(4) +
"><span class=\"slider\"></span></label>";

    buttons += "<h4>Output - GPIO 14</h4><label class=\"switch\"><input
type=\"checkbox\" onchange=\"toggleCheckbox(this)\" id=\"14\" " + outputState(33) +
"><span class=\"slider\"></span></label>";

    buttons += "<h4>Output - GPIO 27</h4><label class=\"switch\"><input
type=\"checkbox\" onchange=\"toggleCheckbox(this)\" id=\"27\" " + outputState(33) +
"><span class=\"slider\"></span></label>";

    return buttons;

  }

  return String();

}

String outputState(int

  output){if(digitalRead(output)

  ){ return "checked";

  }

  else

    { return

    "";

  }

}

void setup(){

  // Serial port for debugging purposes
```

```
Serial.begin(115200);
pinMode(13, OUTPUT);
digitalWrite(13, LOW);
pinMode(12, OUTPUT);
digitalWrite(12, LOW);
pinMode(14, OUTPUT);
digitalWrite(14, LOW);
pinMode(27, OUTPUT);
digitalWrite(27, LOW);
  // Connect to Wi-Fi
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED)
  {delay(1000);
  Serial.println("Connecting to WiFi..");
}
// Print ESP Local IP Address
Serial.println(WiFi.localIP());


// Route for root / web page
server.on("/", HTTP_GET, [](AsyncWebServerRequest
  *request){request->send_P(200, "text/html", index_html,
  processor);
});
// Send a GET request to
<ESP_IP>/update?output=<inputMessage1>&state=<inputMessage2>
server.on("/update", HTTP_GET, [] (AsyncWebServerRequest *request) {
```

```
    String inputMessage1;
    String inputMessage2;
    // GET input1 value on
<ESP_IP>/update?output=<inputMessage1>&state=<inputMessage2>
    if (request->hasParam(PARAM_INPUT_1) && request-
>hasParam(PARAM_INPUT_2)) {
      inputMessage1 = request->getParam(PARAM_INPUT_1)->value();
      inputMessage2 = request->getParam(PARAM_INPUT_2)->value();
      digitalWrite(inputMessage1.toInt(), inputMessage2.toInt());
    }
    else {
      inputMessage1 = "No message sent";
      inputMessage2 = "No message sent";
    }
    Serial.print("GPIO: ");
    Serial.print(inputMessage1);
    Serial.print(" - Set to: ");
    Serial.println(inputMessage2);
    request->send(200, "text/plain", "OK");
  });
  // Start server
  server.begin();
}
void loop() {
}
```

Precautions:

1. Take care about given power supply (12 v)
2. Jumper wires given carefully whenever given circuit connection

Outcome:

LED's, Buzzer and Relay are controlled by using web-based application with ESP32.

Date:

Experiment – 8

Displaying Humidity and Temperature Data on a web-Based Application interfacing with ESP32 board.

Aim: TO interface Displaying Humidity and Temperature Data on a web-Based Application interfacing with ESP32 board.

Hardware Requirements: Development Board, ESP32 Board, DHT11 sensor, 12v Adapter, Power jack, USB cable, Jumper Wires.

Software Requirements: Arduino IDE with supporting ESP32 board software.

Hardware Connections:

1. DHT11 sensor in Development Board DHT11 out pin connected to D33 pin of ESP32 board
2. Power jack is connected to the ESP32 board
3. USB connector is connected to ESP32 board and CPU
4. Connect the 12v power supply to development board
5. Upload the corresponding program into ESP32 board
6. Switch on Power supply

Software Connections:

1. Click on Arduino IDE
2. Click on file
3. Click on New
4. Write a program as per circuit pin connections
5. Click on save
6. Click on verify
7. Click on tools select ESP32 board and select port
8. Click on upload the code into ESP32 board by using USB cable
9. GOTO tools click on serial plotter verify the output in graph format

Source code:

```
#include <WiFi.h>

#include <Wire.h>
```

```
#include "DHT.h"
#define DHTPIN 33
float temp;
float humi;
// Uncomment whatever type you're using!
#define DHTTYPE DHT11      // DHT 11
// Replace with your network credentials
const char* ssid     = "STTMANI";
const char* password = "hailucky123,./";
// Set web server port number to 80
WiFiServer server(80);
// Variable to store the HTTP request
String header;
DHT dht(DHTPIN, DHTTYPE);
void setup()
{
  Serial.begin(115200);
  dht.begin();
  bool status;
  // Connect to Wi-Fi network with SSID and password
  Serial.print("Connecting to ");
  Serial.println(ssid);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED)
    {delay(1000);
```

```
    Serial.print(".");
  }
  // Print local IP address and start web server
  Serial.println("");
  Serial.println("WiFi connected.");
  Serial.println("IP address: ");
  Serial.println(WiFi.localIP());
  server.begin();
}
void loop()
{
  humi = dht.readHumidity();
  temp = dht.readTemperature();
  WiFiClient client = server.available(); // Listen for incoming clients
  if (client)
  { // If a new client connects,
    Serial.println("New Client.");          // print a message out in the serial port
    String currentLine = "";                // make a String to hold incoming data from the
client
    while (client.connected()) {            //  loop while the client's  connected
      if (client.available()) {             // if there's bytes to read from the client,
      char c = client.read();               // read a byte, then
        Serial.write(c);                    // print it out the serial monitor
        header += c;
        if (c == '\n') {                    // if the byte is a newline character
```

```
            // if the current line is blank, you got two newline characters in a row.
            // that's the end of the client HTTP request, so send a response:
            if (currentLine.length() == 0) {
              // HTTP headers always start with a response code (e.g. HTTP/1.1 200 OK)
              // and a content-type so the client knows what's coming, then a blank line:
              client.println("HTTP/1.1 200 OK");
              client.println("Content-type:text/html");
              client.println("Connection: close");
              client.println();
              // Display the HTML web page
              client.println("<!DOCTYPE html><html>");
              client.println("<head><meta name=\"viewport\" content=\"width=device-width,
initial-scale=1\">");
              client.println("<link rel=\"icon\" href=\"data:,\">");
              // CSS to style the table
              client.println("<style>body { text-align: center; font-family: \"Trebuchet MS\",
Arial;}");
              client.println("table { border-collapse: collapse; width:35%; margin-left:auto;
margin-right:auto; }");
              client.println("th { padding: 12px; background-color: #0043af; color: white; }");
              client.println("tr { border: 1px solid #ddd; padding: 12px; }");
              client.println("tr:hover { background-color: #bcbcbc; }");
              client.println("td { border: none; padding: 12px; }");
              client.println(".sensor { color:white; font-weight: bold; background-color:
#bcbcbc; padding: 1px; }");
              // Web Page Heading
              client.println("</style></head><body><h1>DHT11 Using ESP32</font></h1>");
```

```
            client.println("<table><tr><th>SENSOR</th><th>VALUE</th></tr>");
            client.println("<tr><td>Temperature</td><td><span class=\"sensor\">");
            client.println(temp);
            client.println("</span></td></tr>");
            client.println("<tr><td>Humidity</td><td><span class=\"sensor\">");
            client.println(humi);
            client.println("</span></td></tr>");
            client.println("</body></html>");
            // The HTTP response ends with another blank line
            client.println();
            // Break out of the while loop
            break;
          } else { // if you got a newline, then clear currentLine
            currentLine = "";
          }
        } else if (c != '\r') { // if you got anything else but a carriage return character,
          currentLine += c;      // add it to the end of the currentLine
        }
      }
    }
    // Clear the header variable
    header = "";
    // Close the connection
    client.stop();
    Serial.println("Client disconnected.");
```

```
        Serial.println("");
      }
    }
```

Precautions:

1. Take care about given power supply (12 v)
2. Jumper wires given carefully whenever given circuit connection

Outcome:

DHT11 is controlled by using web-based application with ESP32.

Date:

Experiment – 9

Controlling LED ON/OFF MQTT Web Based Interfacing with ESP32

Aim: To interface controlling LED ON/OFF MQTT based interfacing with ESP32 board.

Hardware Requirements: Development Board, LED, ESP32 board, switch, 12v Adapter, Power jack, USB cable, Jumper Wires.

Software Requirements: Arduino IDE with supporting ESP32 board software.

Hardware Connections:

1. LED in Development Board LED1 pin connected to D27 pin of ESP32 board
2. Power jack is connected to the ESP32 board
3. USB connecter is connected to ESP32 board and CPU
4. Connect the 12v power supply to development board
5. Upload corresponding program into ESP32 board
6. Switch on power supply

Software Connections:

1. Click on Arduino IDE
2. Click on file
3. Click on New
4. Write a program as per circuit pin connections
5. Click on save
6. Click on verify
7. Click on tools select ESP32 board and select port
8. Click on upload the code into ESP32 board by using USB cable
9. GOTO tools click on serial plotter verify the output in graph format

Source code:

```
#include <WiFi.h>

#include "Adafruit_MQTT.h"

#include "Adafruit_MQTT_Client.h"

/************************ WiFi Access Point
*********************************/
```

```
#define WLAN_SSID       "STTMANI"

#define WLAN_PASS       "hailucky123,./"

/************************* Adafruit.io Setup
*********************************/

#define AIO_SERVER      "io.adafruit.com"

#define AIO_SERVERPORT  1883                   // use 8883 for SSL

#define AIO_USERNAME    "raviteja27"

#define AIO_KEY         "aio_uuAh70DL5k22E2Ss7PV3IJortZoE"

/************ Global State (you don't need to change this!) ******************/

// Create an ESP8266 WiFiClient class to connect to the MQTT server.

WiFiClient client;

// or... use WiFiFlientSecure for SSL

//WiFiClientSecure client;

// Setup the MQTT client class by passing in the WiFi client and MQTT server and login
details.

Adafruit_MQTT_Client mqtt(&client, AIO_SERVER, AIO_SERVERPORT,
AIO_USERNAME, AIO_KEY);

/****************************** Feeds
***************************************/

// Setup a feed called 'photocell' for publishing.

// Notice MQTT paths for AIO follow the form: <username>/feeds/<feedname>

// Setup a feed called 'onoff' for subscribing to changes.

Adafruit_MQTT_Subscribe onoffbutton = Adafruit_MQTT_Subscribe(&mqtt,
AIO_USERNAME "/feeds/ON_OFF");

/*************************** Sketch Code
************************************/

// Bug workaround for Arduino 1.6.6, it seems to need a function declaration
```

```
// for some reason (only affects ESP8266, likely an arduino-builder bug).
void MQTT_connect();
void setup()
  { Serial.begin(115200);
  delay(10);
  pinMode(27,OUTPUT);
  Serial.println(F("Adafruit MQTT demo"));
  // Connect to WiFi access point.
  Serial.println(); Serial.println();
  Serial.print("Connecting to ");
  Serial.println(WLAN_SSID);
  WiFi.begin(WLAN_SSID, WLAN_PASS);
  while (WiFi.status() != WL_CONNECTED)
    {delay(500);
   Serial.print(".");
  }
  Serial.println();
  Serial.println("WiFi connected");
  Serial.println("IP address: "); Serial.println(WiFi.localIP());
  // Setup MQTT subscription for onoff feed.mqtt.subscribe(&onoffbutton);
}
uint32_t x=0;
void loop() {
  // Ensure the connection to the MQTT server is alive (this will make the first
```

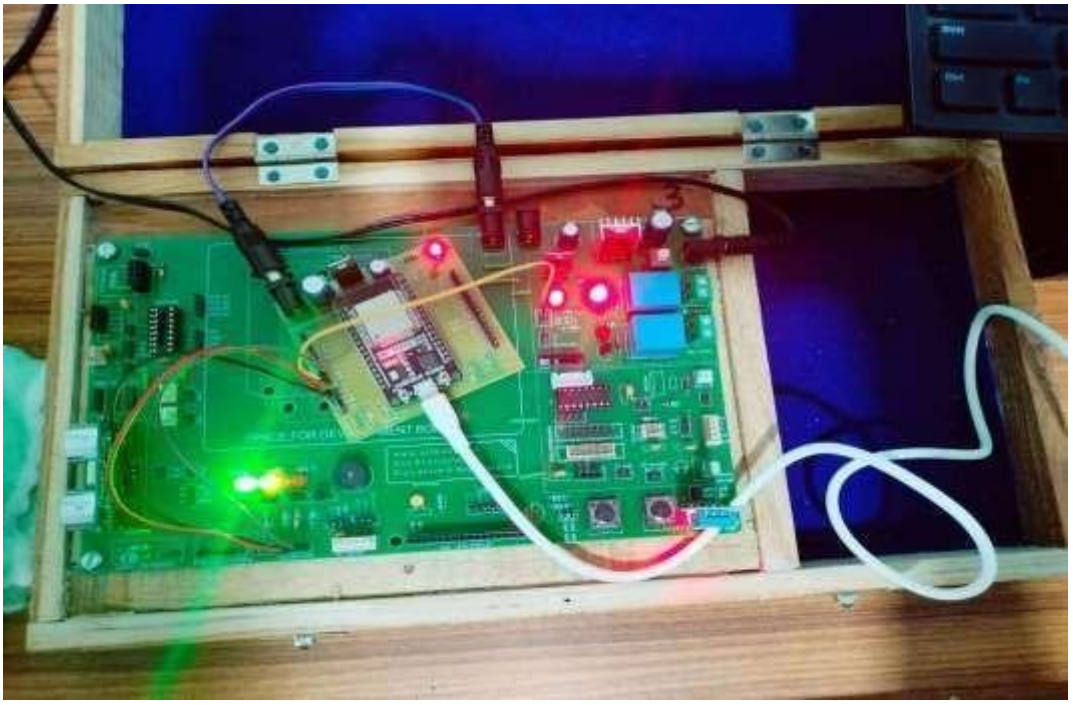
```
  // connection and automatically reconnect when disconnected). See the MQTT_connect
  // function definition further below.
  MQTT_connect();
  // this is our 'wait for incoming subscription packets' busy subloop
  // try to spend your time here
  Adafruit_MQTT_Subscribe *subscription;
  while ((subscription = mqtt.readSubscription(5000)))
    {if (subscription == &onoffbutton)
    { Serial.print(F("Got: "));
      String status = (char *)onoffbutton.lastread;
      Serial.println(status);
      if(status.equals("ON")){digitalWrite(27,HIGH);}
      else if(status.equals("OFF")){digitalWrite(27,LOW);}
    }
  }
}
// Function to connect and reconnect as necessary to the MQTT server.
// Should be called in the loop function and it will take care if connecting.
void MQTT_connect() {
  int8_t ret;
  // Stop if already connected.
  if (mqtt.connected())
  { return;
  }
  Serial.print("Connecting to MQTT... ");
```

```
  uint8_t retries = 3;
  while ((ret = mqtt.connect()) != 0) { // connect will return 0 for connected
      Serial.println(mqtt.connectErrorString(ret));
      Serial.println("Retrying MQTT connection in 5 seconds...");
      mqtt.disconnect();
      delay(5000); // wait 5 seconds
      retries--;
      if (retries == 0) {
        // basically die and wait for WDT to reset me
        while (1);
      }
  }
  Serial.println("MQTT Connected!");
}
```

Precautions:

1. Take care about given power supply (12 v)
2. Jumper wires given carefully whenever given circuit connection

Outcome:



LED ON-OFF is controlled by using MQTT server with ESP32.

Date:

Experiment - 10

DHT11 Sensor Interfacing with ESP32 and Send Temperature & Humidity Data to MQTT Server.

Aim: To interface DHT11 sensor interfacing with ESP32 and send Temperature & Humidity data to MQTT server.

Hardware Requirements: Development board, USB cable, ESP32 board, DHT11 sensor, 12v Adapter, Power jack, Jumper Wires.

Software Requirements: Arduino IDE with supporting ESP32 board software.

Hardware Connections:

1. DHT11 sensor in Development Board DHT11 out pin connected to D27 of ESP board
2. Power jack is connected to the ESP32 board
3. USB connector is connected to ESP32 board and CPU
4. Connect the 12v power supply to development board
5. Upload corresponding program into ESP32 board
6. Switch on power supply

Software Connections:

1. Click on Arduino IDE
2. Click on file
3. Click on New
4. Write a program as per circuit pin connections
5. Click on save
6. Click on verify
7. Click on tools select ESP32 board and select port
8. Click on upload the code into ESP32 board by using USB cable
9. GOTO tools click on serial plotter verify the output in graph format

Source code:

```
#include <SimpleDHT.h>              // Data ---> D3 VCC ---> 3V3 GND ---> GND

#include <WiFi.h>

#include "Adafruit_MQTT.h"
```

```
#include "Adafruit_MQTT_Client.h"

// WiFi parameters

#define WLAN_SSID       "computerlab"

#define WLAN_PASS       "tech@8033"

// Adafruit IO

#define AIO_SERVER      "io.adafruit.com"

#define AIO_SERVERPORT 1883

#define AIO_USERNAME    "kumar_stt"

#define AIO_KEY         "aio_bVFo03c71y4gEDD5ZOEAT67zTpCT"

WiFiClient client;

// Setup the MQTT client class by passing in the WiFi client and MQTT server and login details.

Adafruit_MQTT_Client mqtt(&client, AIO_SERVER, AIO_SERVERPORT, AIO_USERNAME, AIO_KEY);

Adafruit_MQTT_Publish Temperature1 = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME "/feeds/Temperature Value");

Adafruit_MQTT_Publish Humidity1 = Adafruit_MQTT_Publish(&mqtt, AIO_USERNAME "/feeds/Humidity Value");

int pinDHT11 = 27;

SimpleDHT11 dht11(pinDHT11);

byte hum = 0; //Stores humidity value

byte temp = 0; //Stores temperature value

void setup()

{

  Serial.begin(115200);

  Serial.println(F("Adafruit IO Example"));

  // Connect to WiFi access point.

  Serial.println(); Serial.println();

  delay(10);

  Serial.print(F("Connecting to "));
```

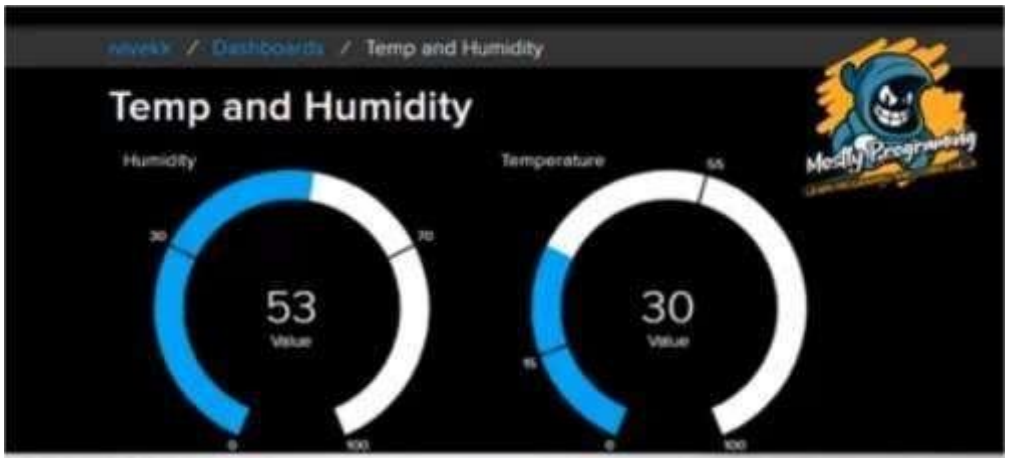
```
  Serial.println(WLAN_SSID);
  WiFi.begin(WLAN_SSID, WLAN_PASS);
  while (WiFi.status() != WL_CONNECTED)
    {delay(500);
   Serial.print(F("."));
  }
  Serial.println();
  Serial.println(F("WiFi connected"));
  Serial.println(F("IP address: "));
  Serial.println(WiFi.localIP());
  // connect to adafruit io
  connect();
}
// connect to adafruit io via MQTT
void connect() {
  Serial.print(F("Connecting to Adafruit IO... "));
  int8_t ret;
  while ((ret = mqtt.connect()) != 0)
    {switch (ret) {
      case 1: Serial.println(F("Wrong protocol")); break;
      case 2: Serial.println(F("ID  rejected"));  break;
      case 3: Serial.println(F("Server unavail")); break;
      case 4: Serial.println(F("Bad user/pass")); break;
      case 5: Serial.println(F("Not authed")); break;
      case 6: Serial.println(F("Failed to subscribe")); break;default:
      Serial.println(F("Connection failed")); break;
    }
    if(ret >= 0)
```

```
    mqtt.disconnect();
   Serial.println(F("Retrying connection..."));
   delay(10000);
  }
  Serial.println(F("Adafruit IO Connected!"));
}
void loop() {
  // ping adafruit io a few times to make sure we remain connectedif(!
  mqtt.ping(3)) {
   // reconnect to adafruit ioif(!
   mqtt.connected()) connect();
  }
  dht11.read(&temp, &hum, NULL);
  Serial.print((int)temp); Serial.print(" *C, ");
  Serial.print((int)hum); Serial.println(" H");
  delay(5000);
   if (! Temperature1.publish(temp)) {                    //Publish to Adafruit
     Serial.println(F("Failed"));
   }
     if (! Humidity1.publish(hum)) {                    //Publish to Adafruit
     Serial.println(F("Failed"));
   }
   else
    { Serial.println(F("Sent!")
     );
   }
}
```

Precautions:

1. Take care about given power supply (12 v)
2. Jumper wires given carefully whenever given circuit connection

Outcome:



MQTT Server receives temperature & humidity data (DHT11) with ESP32.

Date:

Experiment - 12

GPS determine the location of Device interfacing with Raspberry pi Pico

Controlling relay state based on ambient light levels using LDR sensor.

Aim: To interface GPS Module using Raspberry pi Pico board.

Hardware Requirements: Development Board, USB cable, Raspberry pi Pico board, 12v Adapter, GPS Module, Power jack, Jumper Wires.

Software Requirements: Thonny IDE with supporting Raspberry pi Pico software board.

Hardware Connections:

1. GPS out pin is connected to Raspberry pi Pico GPIO pin1
2. Power jack is connected to Raspberry pi Pico
3. Insert GPS and fit into GPS pins Jack
4. USB connector is connected to Raspberry pi Pico to CPU/Laptop
5. Connect the 12v power supply to development board
6. Switch on the power supply

Software Connections:

1. Click on Thonny IDE
2. Click on file
3. Click on New
4. Write a program as per circuit pin connections
5. Click on save
6. Click on Run, select interpreter and select Micro-python (Raspberry pi Pico), select port then click OK
7. Click on Run

Source code:

```
from machine import Pin, UART, I2C
import utime, time
```

```
gpsModule = UART(0, baudrate=9600, tx=Pin(0), rx=Pin(1))
print(gpsModule)

buff = bytearray(255)

TIMEOUT = False
FIX_STATUS = False

latitude = ""
longitude = ""
satellites = ""
GPStime = ""

def getGPS(gpsModule):
    global FIX_STATUS, TIMEOUT, latitude, longitude, satellites, GPStime

    timeout = time.time() + 8
    while True:
        gpsModule.readline()
        buff = str(gpsModule.readline())
        parts = buff.split(',')

        if (parts[0] == "b'$GPGGA" and len(parts) == 15):
            if(parts[1] and parts[2] and parts[3] and parts[4] and parts[5] and parts[6] and parts[7]):
                print(buff)

                latitude = convertToDegree(parts[2])
                if (parts[3] == 'S'):
                    latitude = -latitude
                longitude = convertToDegree(parts[4])
                if (parts[5] == 'W'):
                    longitude = -longitude
                satellites = parts[7]
                GPStime = parts[1][0:2] + ":" + parts[1][2:4] + ":" + parts[1][4:6]
                FIX_STATUS = True
                break

        if (time.time() > timeout):
            TIMEOUT = True
            break
        utime.sleep_ms(500)

def convertToDegree(RawDegrees):

    RawAsFloat = float(RawDegrees)
    firstdigits = int(RawAsFloat/100)
    nexttwodigits = RawAsFloat - float(firstdigits*100)
```

```
Converted = float(firstdigits + nexttwodigits/60.0)
Converted = '{0:.6f}'.format(Converted)
return str(Converted)

while True:

getGPS(gpsModule)

if(FIX_STATUS == True):
    print("Printing GPS data...")
    print(" ")
    print("Latitude: "+latitude)
    print("Longitude: "+longitude)
    print("Satellites: " +satellites)
    print("Time: "+GPStime)
    print(".....")

    FIX_STATUS = False

if(TIMEOUT == True):
    #print("No GPS data is found.")
    TIMEOUT = False
```

Precautions:

1. Take care about given power supply (12 v)
2. Jumper wires given carefully whenever given circuit connection

Outcome:

GPS determine the location of Device with Raspberry pi Pico.